

MATH00041: Introduction to Programming

Matplotlib

Lecture 9

Archontis Giannakidis, PhD

Outline

What is matplotlib?

Matplotlib is a Python-based low-level graph plotting library that enables users to create visualisations with ease. The library was created by John D. Hunter and is open source, which means it can be used freely.

Matplotlib is mainly written in Python, with some segments written in C, Objective-C, and JavaScript to ensure compatibility across different platforms.

Where is the matplotlib Codebase?

The source code for Matplotlib is located at this GitHub repository

<https://github.com/matplotlib/matplotlib>

Installation of matplotlib

If you have Python and PIP already installed on a system, then installation of Matplotlib is straightforward.

Install it using this command:

```
C:\Users\Your Name>pip install matplotlib
```

Import matplotlib

Once Matplotlib is installed, import it in your applications by adding the import module statement:

```
import matplotlib
```

Checking matplotlib Version

The version string is stored under `__version__` attribute.

```
import matplotlib

print(matplotlib.__version__)
```

❖ **Note:** two underscore characters are used in `__version__`.

Pyplot

Most of the Matplotlib utilities lies under the `pyplot` submodule, and are usually imported under the `plt` alias:

```
import matplotlib.pyplot as plt
```

Now, the Pyplot package can be referred to as `plt`.

Example:

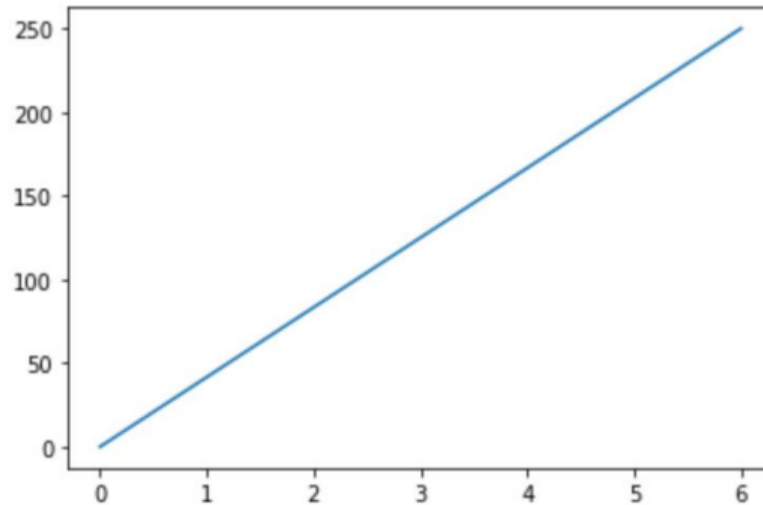
Draw a line in a diagram from position (0,0) to position (6,250):

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
xpoints = np.array([0, 6])  
ypoints = np.array([0, 250])
```

```
plt.plot(xpoints, ypoints)  
plt.show()
```

```
>>
```



Plotting x and y points

To draw points (markers) in a diagram, we use the `plot()` function. The `plot()` function default connects the points with a line. This function requires two parameters to specify the points on the **x-axis** and **y-axis**, respectively. For example, if we want to plot a line from point (1, 3) to (8, 10), we need to pass two arrays, [1, 8] for x-coordinates and [3, 10] for y-coordinates, to the `plot()` function.

Example:

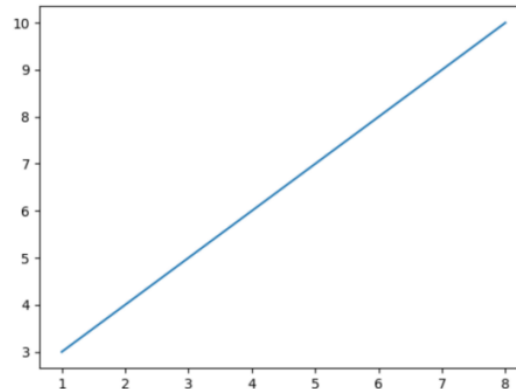
Draw a line in a diagram from position (1, 3) to position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np
```

```
xpoints = np.array([1, 8])
ypoints = np.array([3, 10])
```

```
plt.plot(xpoints, ypoints)
plt.show()
```

>>



❖ **Note:** The x-axis is the horizontal axis. The y-axis is the vertical axis.

Plotting Without Line

To plot only the markers, you can use the shortcut string notation parameter 'o', which means 'rings'.

Example:

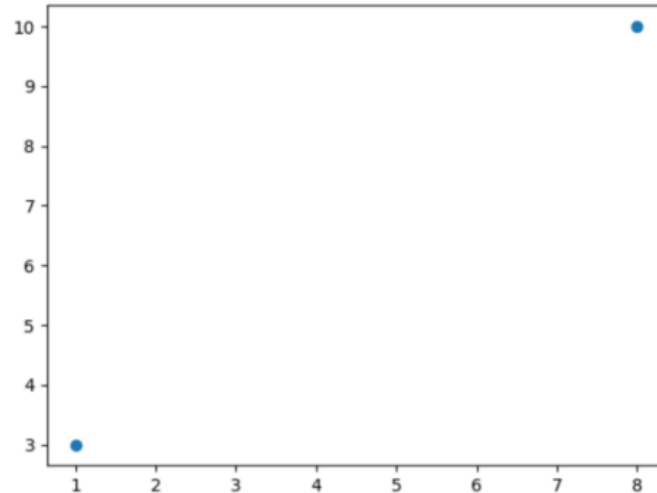
Draw two points in the diagram, one at position (1, 3) and one in position (8, 10):

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
xpoints = np.array([1, 8])  
ypoints = np.array([3, 10])
```

```
plt.plot(xpoints, ypoints, 'o')  
plt.show()
```

>>



Multiple Points

You can plot as many points as you like, make sure you have the same number of points on both axis.

Example:

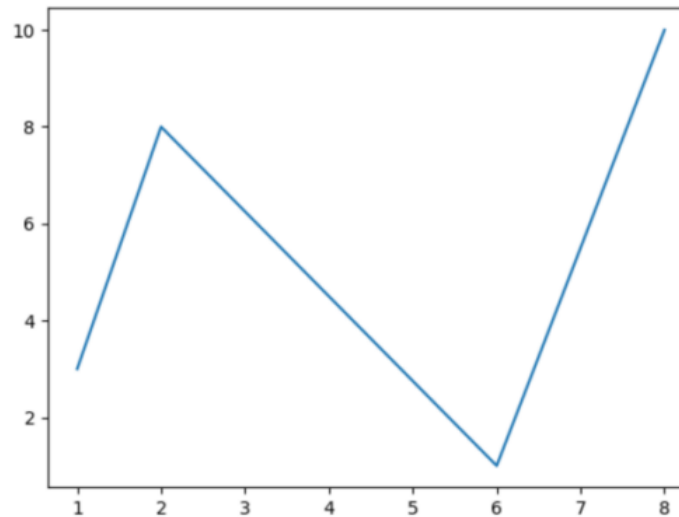
Draw a line in a diagram from position (1, 3) to (2, 8) then to (6, 1) and finally to position (8, 10):

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
xpoints = np.array([1, 2, 6, 8])  
ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(xpoints, ypoints)  
plt.show()
```

>>



Default X-Points

When we create a diagram using the y-points, the x-axis points are assigned default values of 0, 1, 2, 3, and so on, based on the length of the y-points. So, if we exclude the x-points and use the same example as before, the resulting diagram will look like this:

Example:

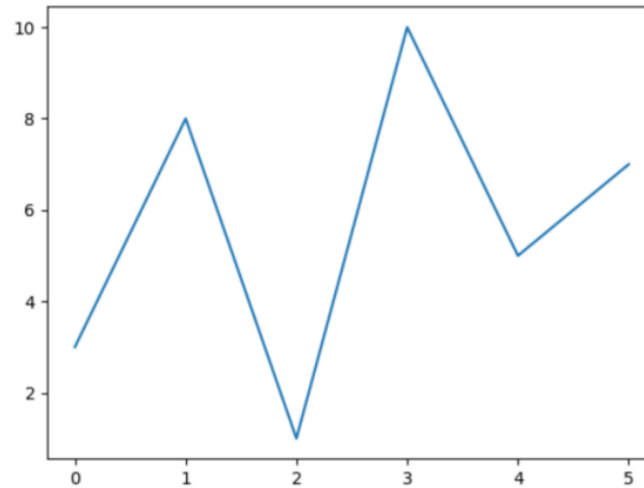
Plotting without x-points:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
ypoints = np.array([3, 8, 1, 10, 5, 7])
```

```
plt.plot(ypoints)
plt.show()
```

>>



❖ **Note:** The x-points in the example above are [0, 1, 2, 3, 4, 5].

Markers

You can use the keyword argument marker to emphasise each point with a specified marker:

Example:

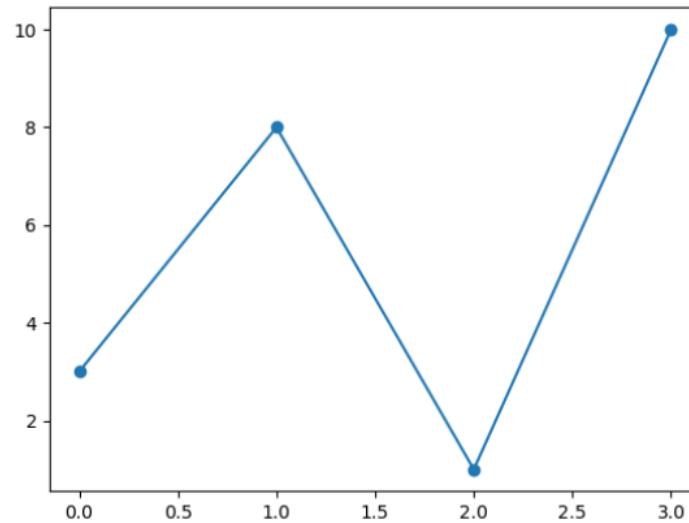
Mark each point with a circle:

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(ypoints, marker = 'o')  
plt.show()
```

```
>>
```



Example:

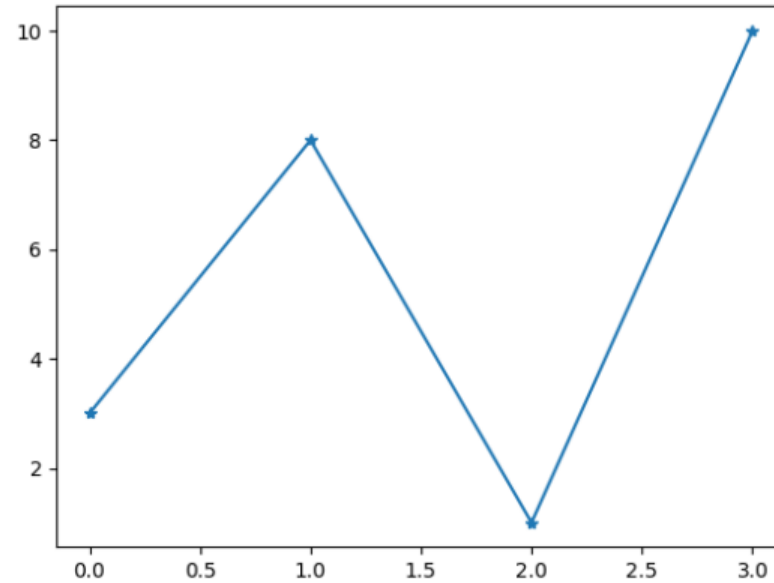
Mark each point with a star:

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(ypoints, marker = '*')  
plt.show()
```

```
>>
```



Marker Reference

You can choose any of these markers:

Marker	Description
o	Circle
*	Star
.	Point
,	Pixel
x	X
X	X (filled)
+	Plus
P	Plus (filled)
s	Square
D	Diamond
d	Diamond (thin)
p	Pentagon
H	Hexagon

h	Hexagon
v	Triangle Down
^	Triangle Up
<	Triangle Left
>	Triangle Right
1	Tri Down
2	Tri Up
3	Tri Left
4	Tri Right
	Vline
—	Hline

Format Strings **fmt**

You can also use the *shortcut string notation* parameter to specify the marker.

This parameter is also called **fmt**, and is written with this syntax:

marker|line|color

Example:

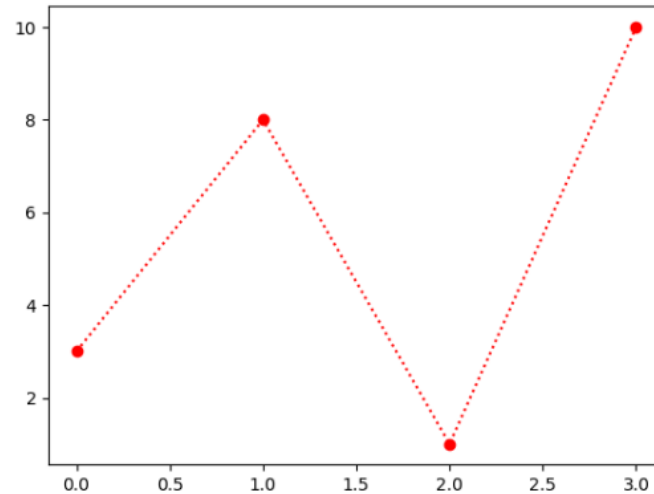
Mark each point with a circle:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(ypoints, 'o:r')
plt.show()
```

>>



Line Reference

The line value can be one of the following:

Line Syntax	Description
-	Solid line
:	Dotted line
--	Dashed line
-.	Dashed/dotted line

❖ **Note:** No line will be plotted if you leave out the line value in the fmt parameter.

Colour Reference

The short colour value can be one of the following:

Color Syntax	Description
r	Red
g	Green
b	Blue
c	Cyan
m	Magenta
y	Yellow
k	Black
w	White

Marker Size

You can use the keyword argument `markersize` or the shorter version, `ms` to set the size of the markers:

Example:

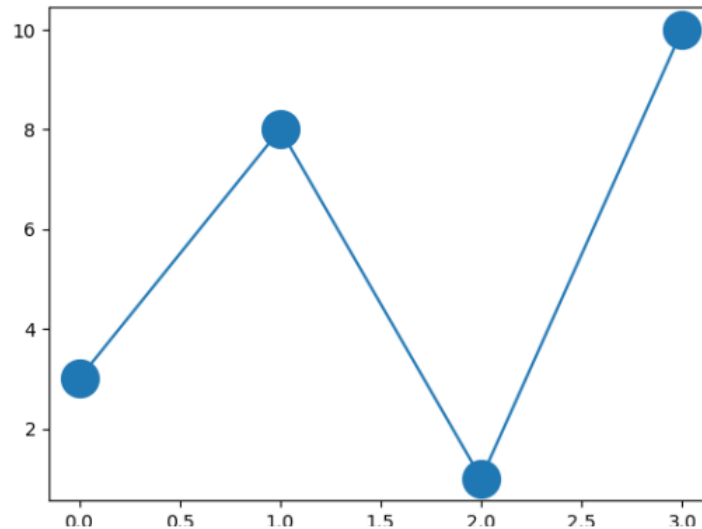
Set the size of the markers to 20:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, marker = 'o', ms = 20)
plt.show()
```

>>



Marker Color

You can use the keyword argument `markeredgecolor` or the shorter `mec` to set the colour of the *edge* of the markers:

Example:

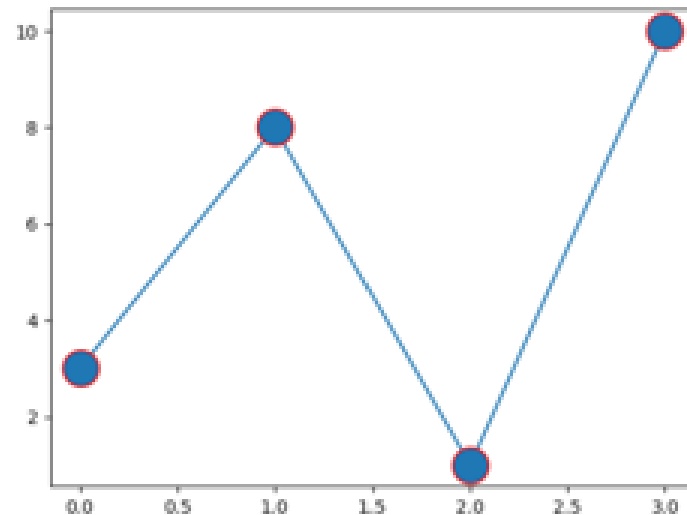
Set the color of the *edge* to red:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, marker = 'o', ms = 20, mec = 'r')
plt.show()
```

```
>>
```



You can use the keyword argument `markerfacecolor` or the shorter `mfc` to set the colour inside the edge of the markers:

Example:

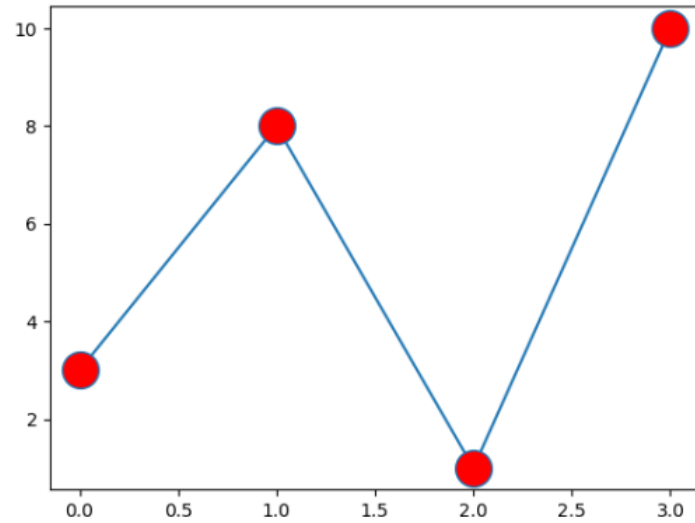
Set the FACE color to red:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(ypoints, marker = 'o', ms = 20, mfc = 'r')
plt.show()
```

```
>>
```



Use both the `mec` and `mfc` arguments to colour the entire marker:

Example:

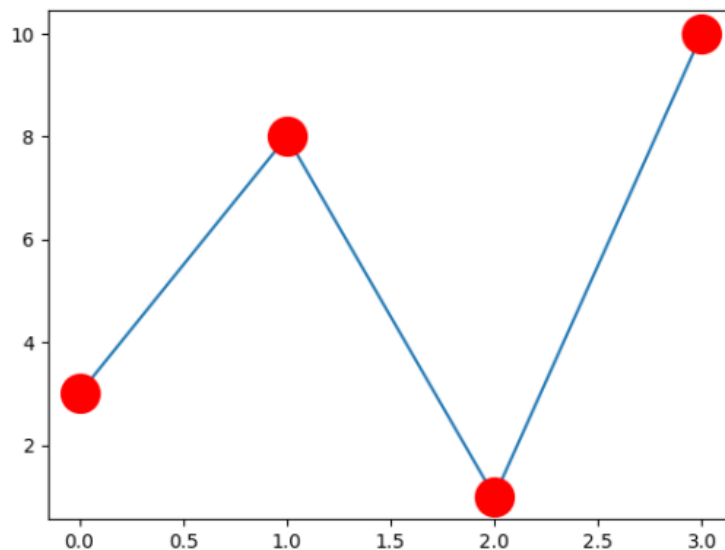
Set the color of both the *edge* and the *face* to red:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(ypoints, marker = 'o', ms = 20, mec = 'r', mfc = 'r')
plt.show()
```

```
>>
```



Matplotlib Line

You can use the keyword argument `linestyle`, or shorter `ls`, to change the style of the plotted line:

Example:

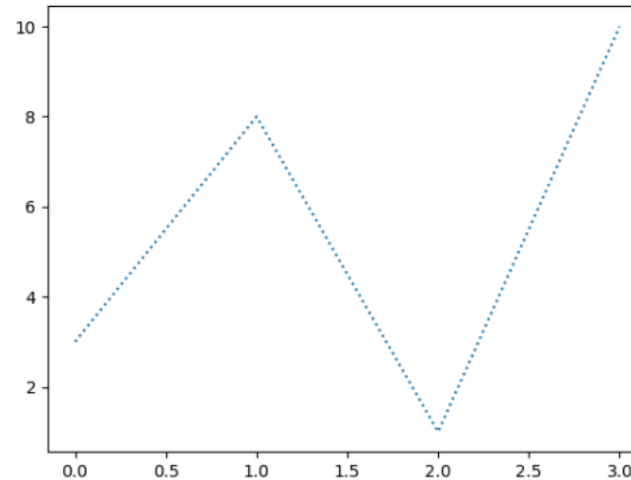
Use a dotted line:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, linestyle = 'dotted')
plt.show()
```

>>



Example:

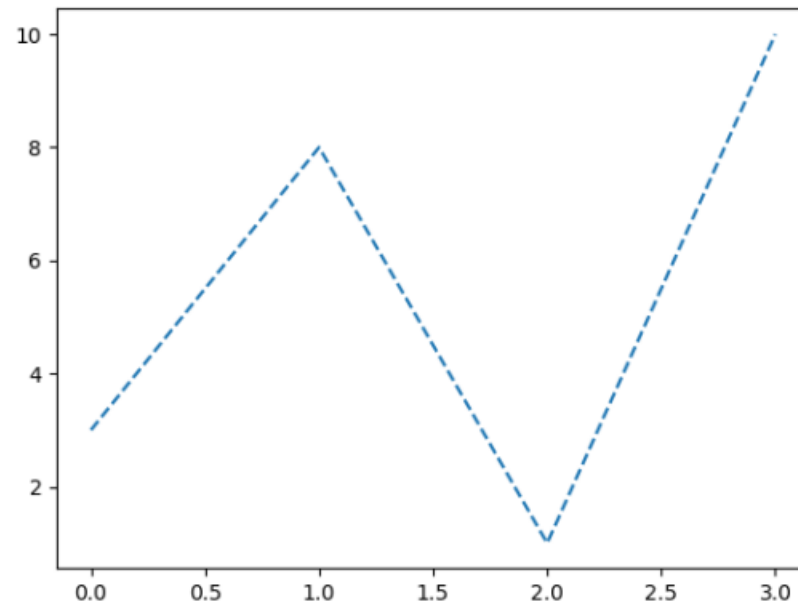
Use a dashed line:

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(ypoints, linestyle = 'dashed')  
plt.show()
```

```
>>
```



Line Color

You can use the keyword argument `color` or the shorter `c` to set the colour of the line:

Example:

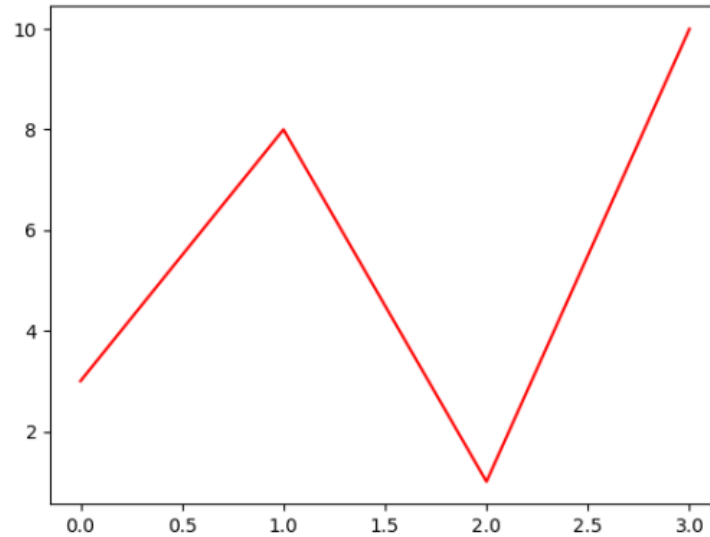
Set the line color to red:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(ypoints, color = 'r')
plt.show()
```

```
>>
```



Line Width

You can use the keyword argument `linewidth` or the shorter `lw` to change the width of the line.

The value is a floating number, in points:

Example:

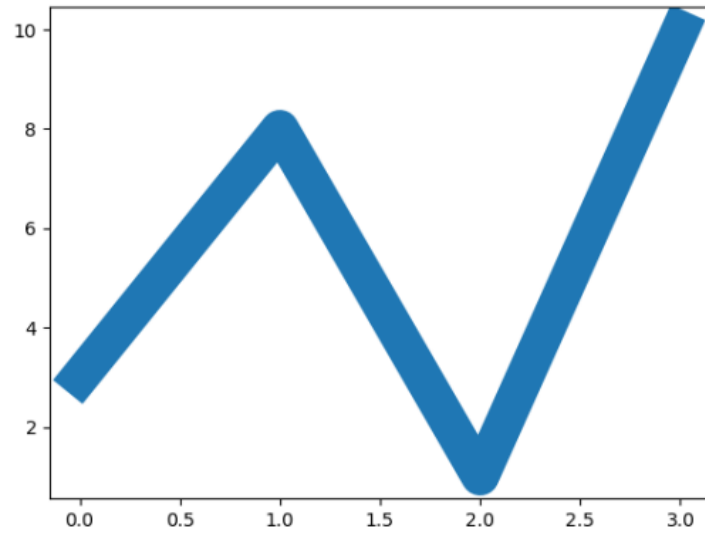
Plot with a 20.5pt wide line:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(ypoints, linewidth = '20.5')
plt.show()
```

>>



Multiple Lines

You can plot as many lines as you like by simply adding more `plt.plot()` functions:

Example:

Draw two lines by specifying a `plt.plot()` function for each line:

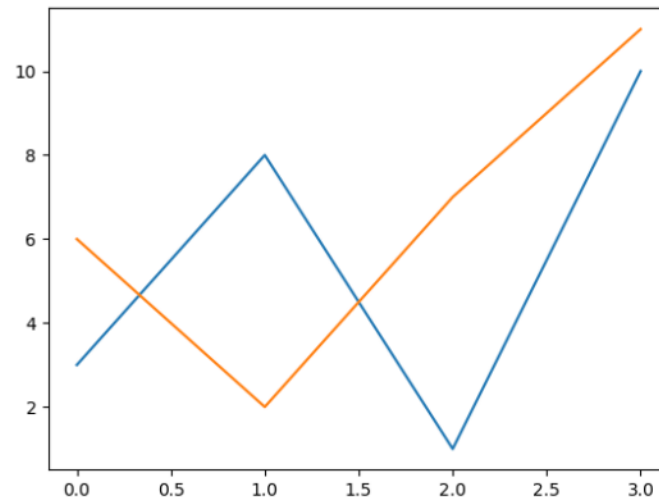
```
import matplotlib.pyplot as plt
import numpy as np
```

```
y1 = np.array([3, 8, 1, 10])
y2 = np.array([6, 2, 7, 11])
```

```
plt.plot(y1)
plt.plot(y2)
```

```
plt.show()
```

```
>>
```



When plotting multiple lines on a graph, you can add the x- and y-axis points for each line in a single '`plt.plot()`' function. In the previous examples, we only specified the y-axis points, which resulted in the x-axis points taking on default values (0, 1, 2, 3). Keep in mind that the x- and y-axis values should be provided in pairs.

Example:

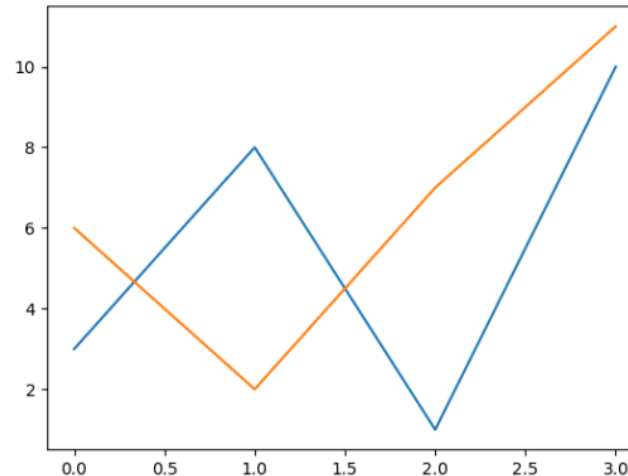
Draw two lines by specifying the x- and y-point values for both lines:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x1 = np.array([0, 1, 2, 3])
y1 = np.array([3, 8, 1, 10])
x2 = np.array([0, 1, 2, 3])
y2 = np.array([6, 2, 7, 11])
```

```
plt.plot(x1, y1, x2, y2)
plt.show()
```

>>



Create Labels for a Plot

With Pyplot, you can use the `xlabel()` and `ylabel()` functions to set a label for the x- and y-axis.

Example:

Add labels to the x- and y-axis:

```
import numpy as np
import matplotlib.pyplot as plt

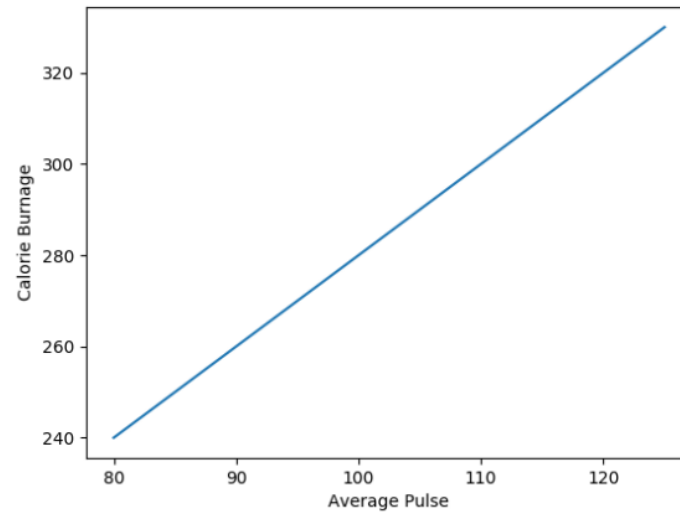
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.plot(x, y)

plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```

>>



Create a Title for a Plot

With Pyplot, you can use the `title()` function to set a title for the plot.

Example:

Add a plot title and labels for the x- and y-axis:

```
import numpy as np
import matplotlib.pyplot as plt

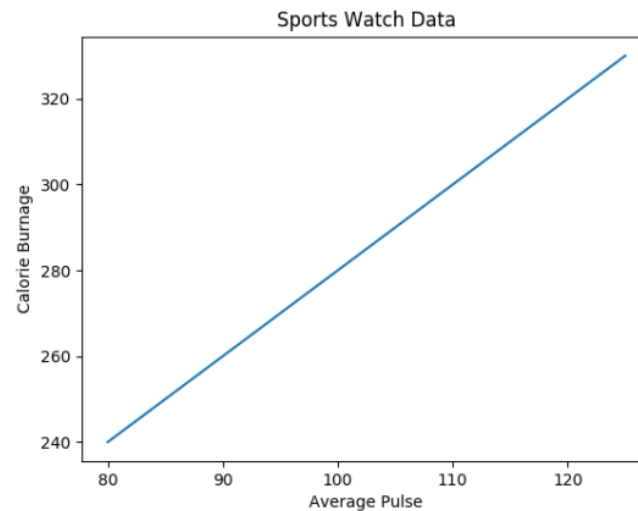
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.plot(x, y)

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```

>>



Set Font Properties for Title and Labels

You can use the `fontdict` parameter in `xlabel()`, `ylabel()`, and `title()` to set font properties for the title and labels.

Example:

Set font properties for the title and labels:

```
import numpy as np
import matplotlib.pyplot as plt

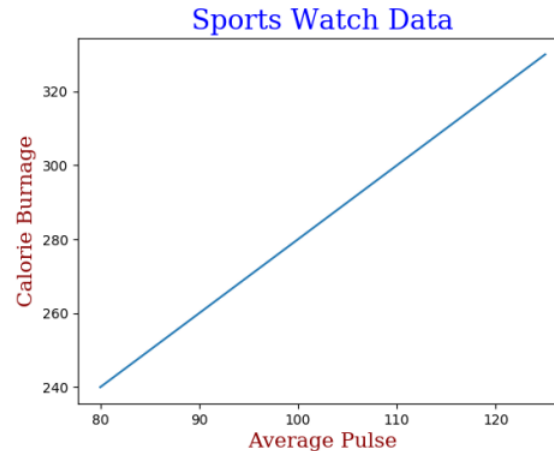
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

font1 = {'family':'serif','color':'blue','size':20}
font2 = {'family':'serif','color':'darkred','size':15}

plt.title("Sports Watch Data", fontdict = font1)
plt.xlabel("Average Pulse", fontdict = font2)
plt.ylabel("Calorie Burnage", fontdict = font2)

plt.plot(x, y)
plt.show()
```

>>



Position the Title

You can use the `loc` parameter in `title()` to position the title.

Legal values are: `'left'`, `'right'`, and `'center'`. Default value is `'center'`.

Example:

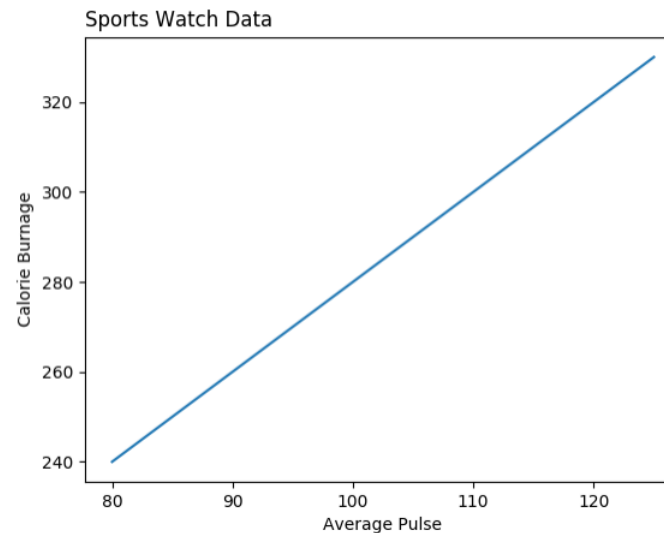
Position the title to the left:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data", loc = 'left')
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x, y)
plt.show()
```



Add Grid Lines to a Plot

With Pyplot, you can use the `grid()` function to add grid lines to the plot.

Example:

Add grid lines to the plot:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

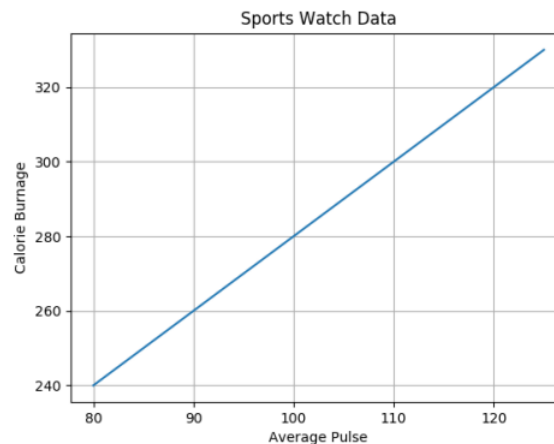
plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x, y)

plt.grid()

plt.show()
```

>>



Specify Which Grid Lines to Display

You can use the `axis` parameter in the `grid()` function to specify which grid lines to display.

Legal values are: 'x', 'y', and 'both'. Default value is 'both'.

Example:

Display only grid lines for the x-axis:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

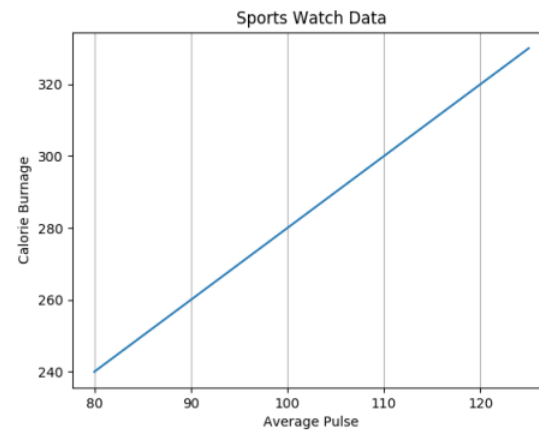
plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x, y)

plt.grid(axis = 'x')

plt.show()
```

>>



Example:

Display only grid lines for the y-axis:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

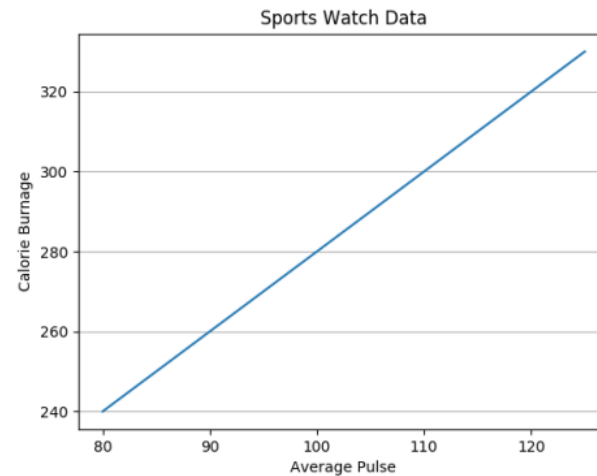
plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x, y)

plt.grid(axis = 'y')

plt.show()
```

>>



Set Line Properties for the Grid

You can also set the line properties of the grid, like this: `grid(color = 'color', linestyle = 'linestyle', linewidth = number)`.

Example:

Set the line properties of the grid:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

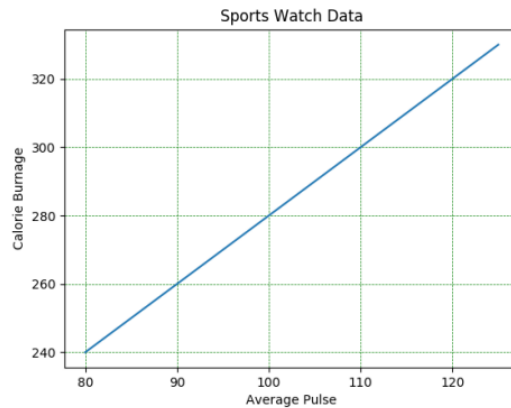
plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x, y)

plt.grid(color = 'green', linestyle = '--', linewidth = 0.5)

plt.show()

>>
```



Display Multiple Plots

With the `subplot()` function, you can draw multiple plots in one figure:

Example:

Draw 2 plots:

```
import matplotlib.pyplot as plt
import numpy as np
```

#plot 1:

```
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
```

```
plt.subplot(1, 2, 1)
plt.plot(x,y)
```

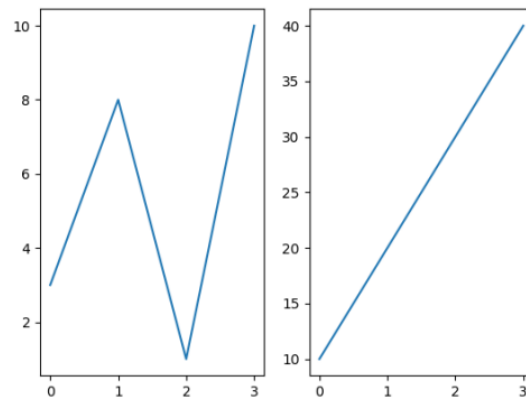
#plot 2:

```
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(1, 2, 2)
plt.plot(x,y)
```

```
plt.show()
```

>>



The subplot() Function

The `subplot()` function arranges plots in a grid, with rows and columns defined by the first two arguments and the plot index by the third.

```
plt.subplot(1, 2, 1)
#the figure has 1 row, 2 columns, and this plot is the first plot.
```

```
plt.subplot(1, 2, 2)
#the figure has 1 row, 2 columns, and this plot is the second plot.
```

For a figure with 2 rows and 1 column, the syntax is:

Example:

Draw 2 plots on top of each other:

```
import matplotlib.pyplot as plt
import numpy as np
```

#plot 1:

```
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
```

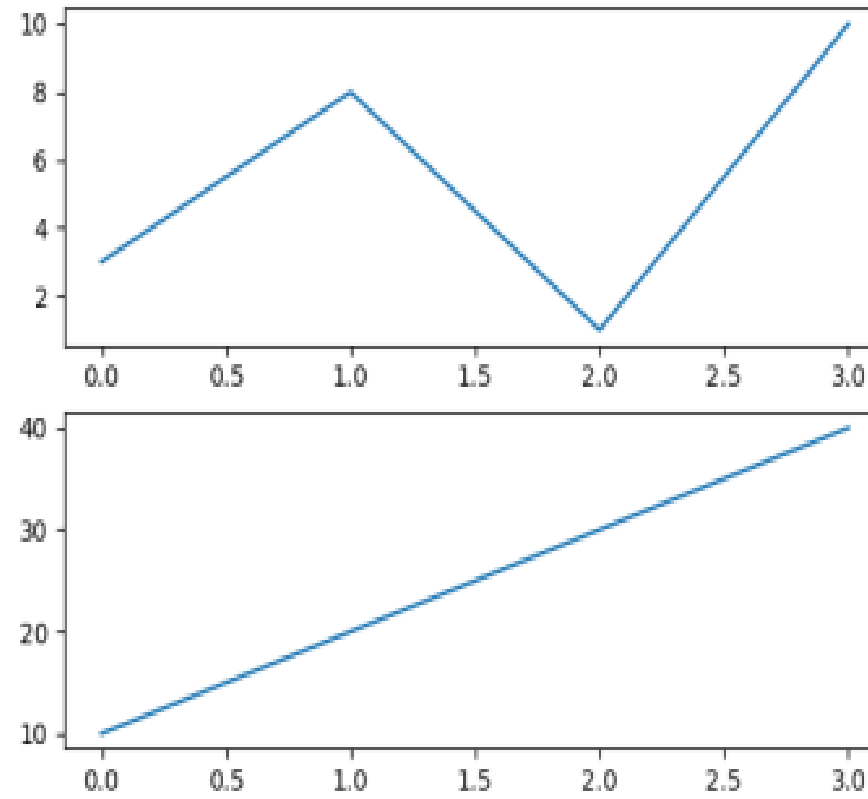
```
plt.subplot(2, 1, 1)
plt.plot(x,y)
```

#plot 2:

```
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(2, 1, 2)
plt.plot(x,y)
```

```
plt.show()
```



Example:

Draw 6 plots:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
```

```
plt.subplot(2, 3, 1)
plt.plot(x,y)
```

```
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(2, 3, 2)
plt.plot(x,y)
```

```
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
```

```
plt.subplot(2, 3, 3)
plt.plot(x,y)
```

>>

```
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(2, 3, 4)
plt.plot(x,y)
```

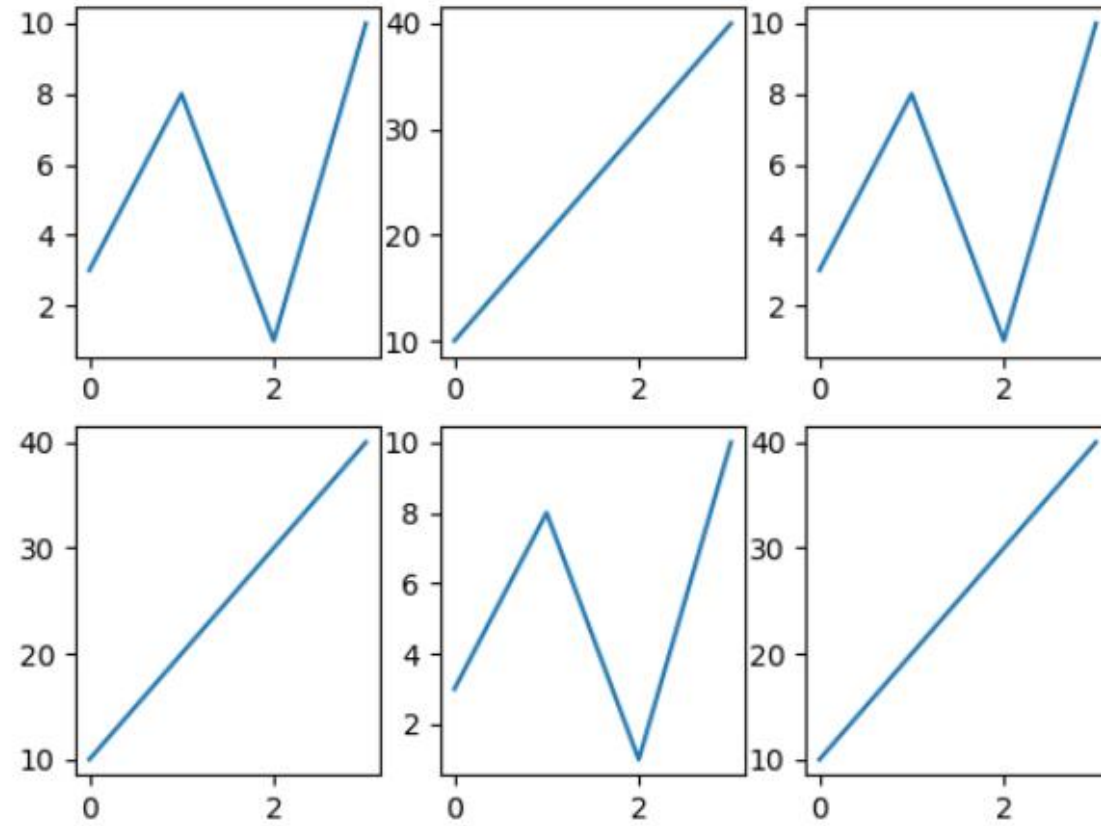
```
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
```

```
plt.subplot(2, 3, 5)
plt.plot(x,y)
```

```
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(2, 3, 6)
plt.plot(x,y)
```

```
plt.show()
```



Subplot Title

You can add a title to each plot with the `title()` function:

Example:

2 plots, with titles:

```
import matplotlib.pyplot as plt
import numpy as np
```

#plot 1:

```
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
```

```
plt.subplot(1, 2, 1)
plt.plot(x,y)
plt.title("SALES")
```

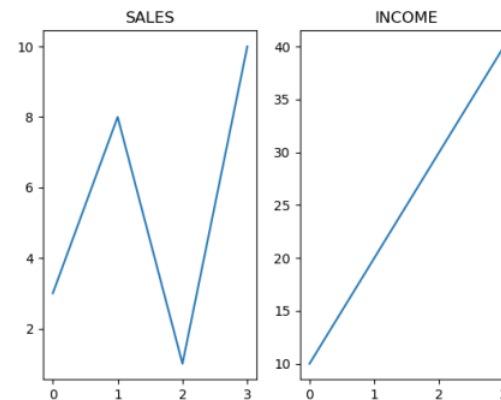
#plot 2:

```
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(1, 2, 2)
plt.plot(x,y)
plt.title("INCOME")
```

```
plt.show()
```

```
>>
```



Super Title

You can add a title to the entire figure with the `suptitle()` function:

Example:

Add a title for the entire figure:

```
import matplotlib.pyplot as plt
import numpy as np
```

#plot 1:

```
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
```

```
plt.subplot(1, 2, 1)
plt.plot(x,y)
plt.title("SALES")
```

#plot 2:

```
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(1, 2, 2)
plt.plot(x,y)
plt.title("INCOME")
```

```
plt.suptitle("MY SHOP")
plt.show()
```

>>



Creating Scatter Plots

To create a scatter plot using Pyplot, you can make use of the `scatter()` function. This function helps to plot a single dot for each observation. In order to plot the dots, you need to provide two arrays of equal length - one for the values on the x-axis and another for the values on the y-axis.

Example:

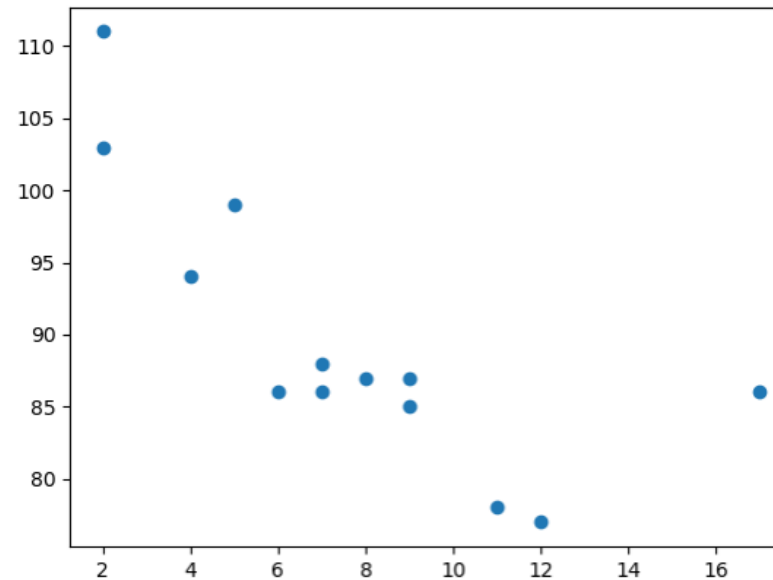
A simple scatter plot:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])

plt.scatter(x, y)
plt.show()
```

>>



Colours

You can set your own colour for each scatter plot with the color or the c argument:

Example:

Draw two scatter plots on the same figure and set your own colour of the markers:

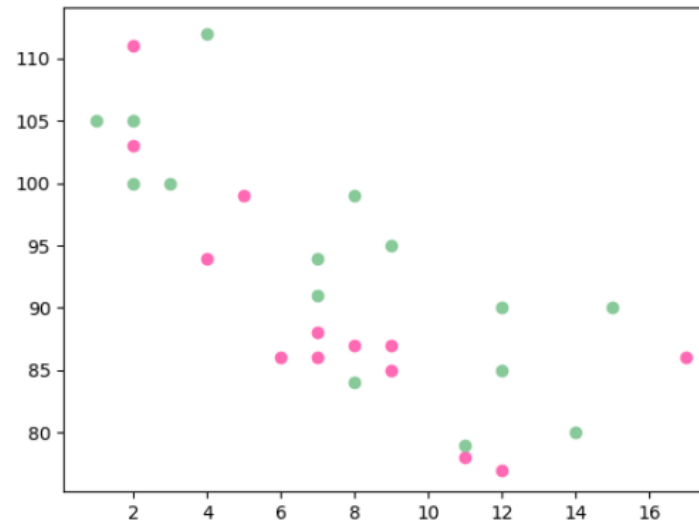
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
plt.scatter(x, y, color = 'hotpink')

x = np.array([2,2,8,1,15,8,12,9,7,3,11,4,7,14,12])
y = np.array([100,105,84,105,90,99,90,95,94,100,79,112,91,80,85])
plt.scatter(x, y, color = '#88c999')

plt.show()
```

>>



Colour Each Dot

You can even set a specific colour for each dot by using an array of colours as value for the c argument:

Example:

Set your own color of the markers:

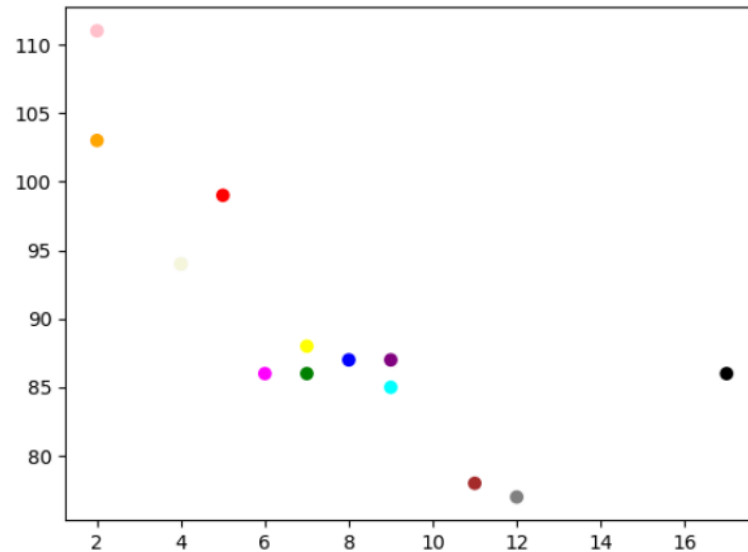
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
colors =
np.array(["red", "green", "blue", "yellow", "pink", "black", "orange", "purple",
"beige", "brown", "gray", "cyan", "magenta"])

plt.scatter(x, y, c=colors)

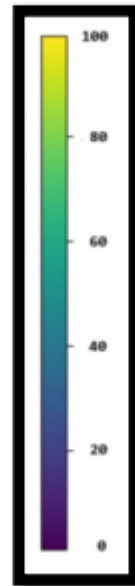
plt.show()
```

>>



ColorMap

The Matplotlib module offers numerous colormaps, where each colour has a value ranging from 0 to 100.



The above colormap is called 'viridis' and as you can see it ranges from 0, which is a purple colour, up to 100, which is a yellow colour.

How to Use the ColorMap

To specify the colormap for your scatter plot in Matplotlib, you can use the 'cmap' keyword argument and set its value to the name of the desired colormap. In this case, you can use the 'viridis' colormap, which is one of the built-in options.

To create a scatter plot, you also need to create an array of values ranging from 0 to 100. Each point in the plot will correspond to one value in the array.

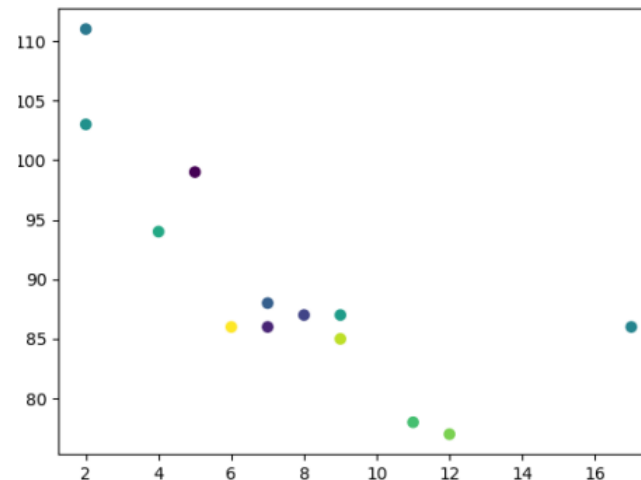
Example:

Create a color array, and specify a colormap in the scatter plot:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
colors =
np.array([0, 10, 20, 30, 40, 45, 50, 55, 60, 70, 80, 90, 100])
plt.scatter(x, y, c=colors, cmap='viridis')
plt.show()
```

>>



You can include the colormap in the drawing by including the `plt.colorbar()` statement:

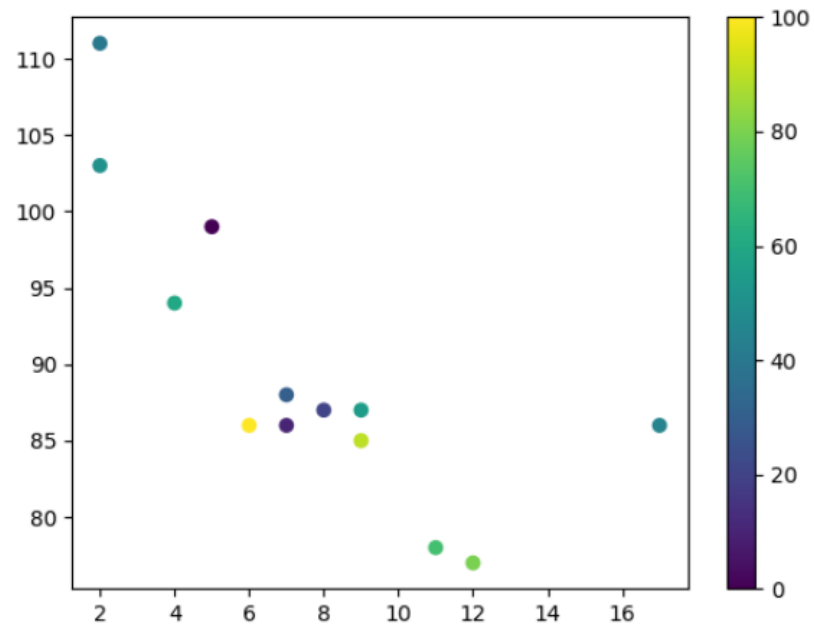
Example:

Include the actual colormap:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
colors =
np.array([0, 10, 20, 30, 40, 45, 50, 55, 60, 70, 80, 90, 100])
plt.scatter(x, y, c=colors, cmap='viridis')
plt.colorbar()
plt.show()
```

>>



Size

You can modify the size of the dots by using the `s` parameter. Make sure that the size array has the same length as `x` and `y` arrays.

Example:

Set your own size for the markers:

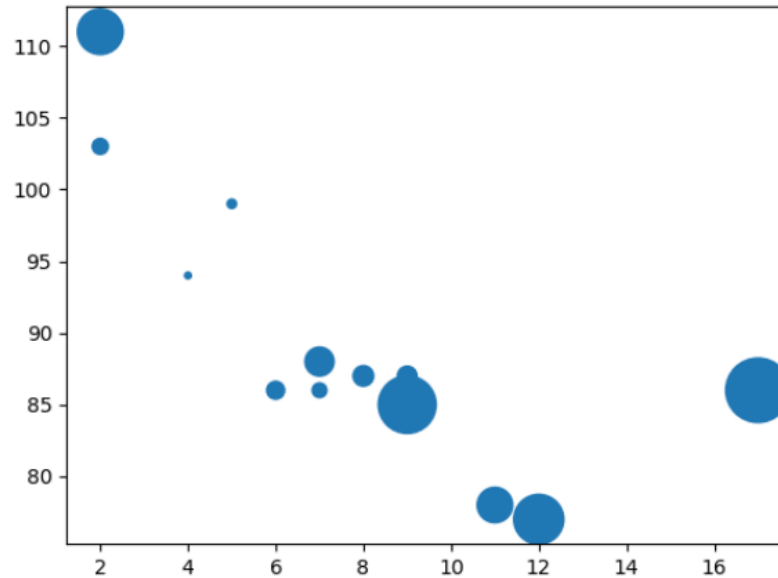
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
sizes = np.array([20,50,100,200,500,1000,60,90,10,300,600,800,75])

plt.scatter(x, y, s=sizes)

plt.show()
```

>>



Alpha

You can adjust dot transparency with the `alpha` argument. Ensure sizes array matches x- and y-axis arrays.

Example:

Set your own transparency for the markers:

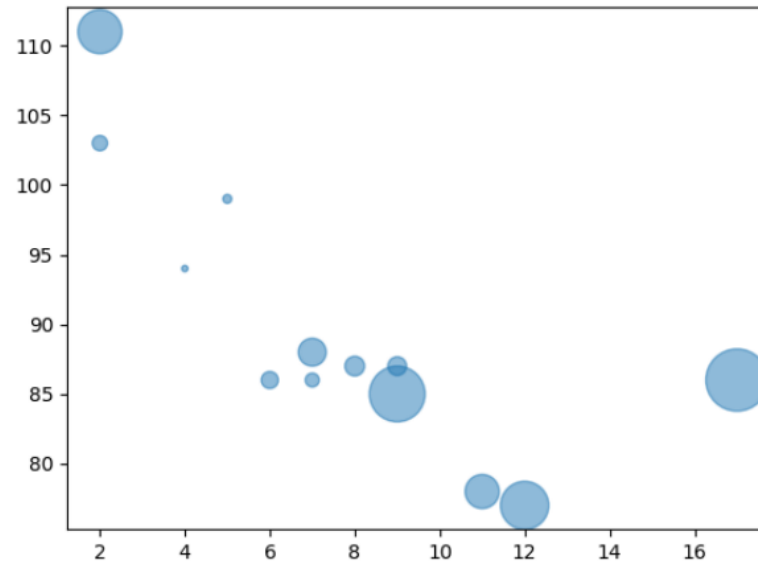
```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
sizes = np.array([20,50,100,200,500,1000,60,90,10,300,600,800,75])
```

```
plt.scatter(x, y, s=sizes, alpha=0.5)
```

```
plt.show()
```

```
>>
```



Matplotlib Bars

With Pyplot, you can use the `bar()` function to draw bar graphs:

Example:

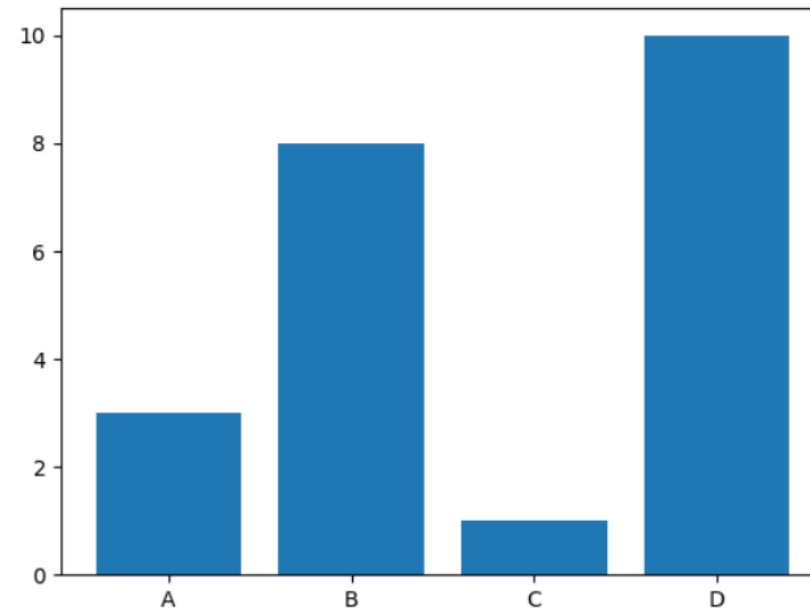
Draw 4 bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y)
plt.show()
```

>>



The function `bar()` uses arrays to define bar layout and data.

Horizontal Bars

If you want the bars to be displayed horizontally instead of vertically, use the `barh()` function:

Example:

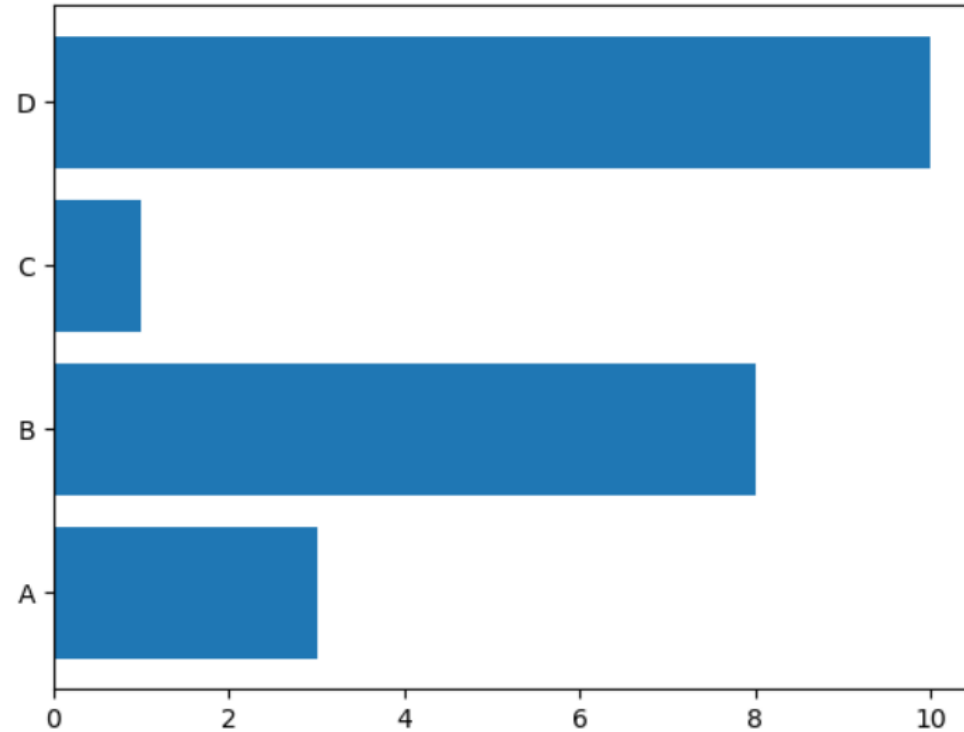
Draw 4 horizontal bars:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])
```

```
plt.barh(x, y)
plt.show()
```

```
>>
```



Bar Color

The `bar()` and `barh()` take the keyword argument `color` to set the color of the bars:

Example:

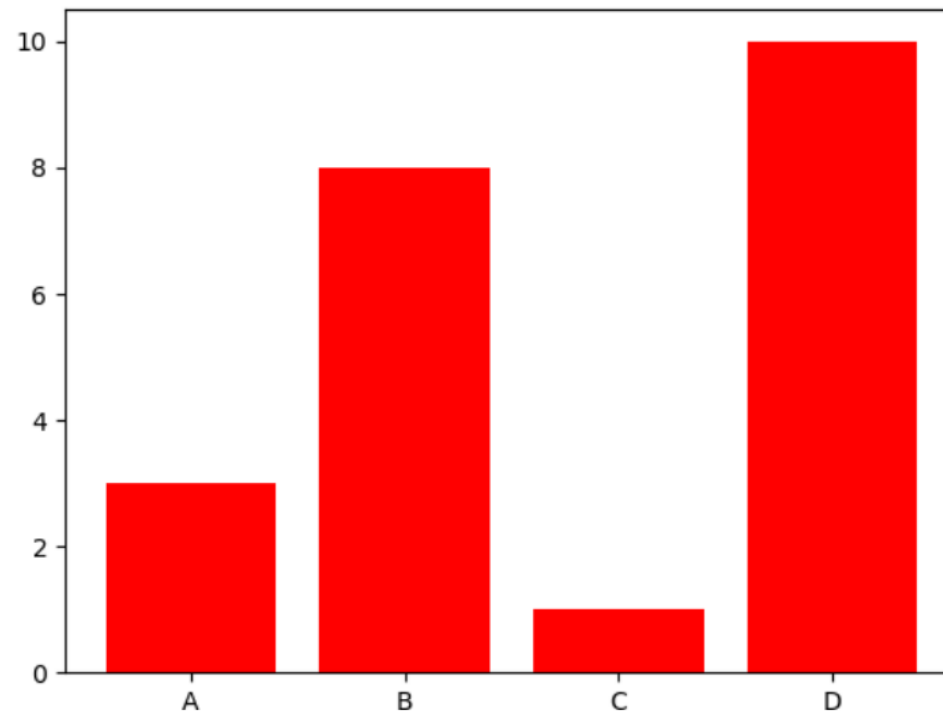
Draw 4 red bars:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])
```

```
plt.bar(x, y, color = "red")
plt.show()
```

```
>>
```



Matplotlib Pie Charts

With Pyplot, you can use the `pie()` function to draw pie charts:

Example:

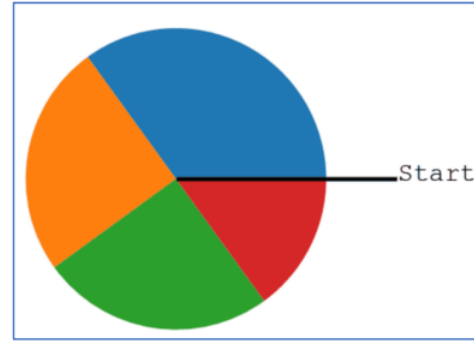
A simple pie chart:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
y = np.array([35, 25, 25, 15])
```

```
plt.pie(y)
plt.show()
```

```
>>
```



As you can see the pie chart draws one piece (called a wedge) for each value in the array (in this case [35, 25, 25, 15]). By default the plotting of the first wedge starts from the x-axis and moves counter clockwise:

- ❖ **Note:** The size of each wedge is determined by comparing the value with all the other values, by using this formula: The value divided by the sum of all values: $x/\text{sum}(x)$

Labels

Add labels to the pie chart with the label parameter. The label parameter must be an array with one label for each wedge:

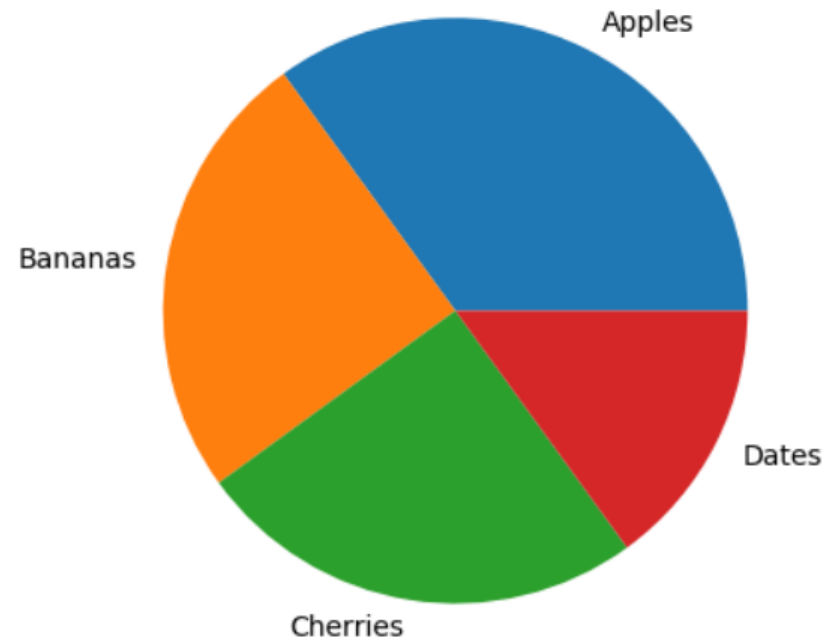
Example:

A simple pie chart with labels:

```
import matplotlib.pyplot as plt
import numpy as np

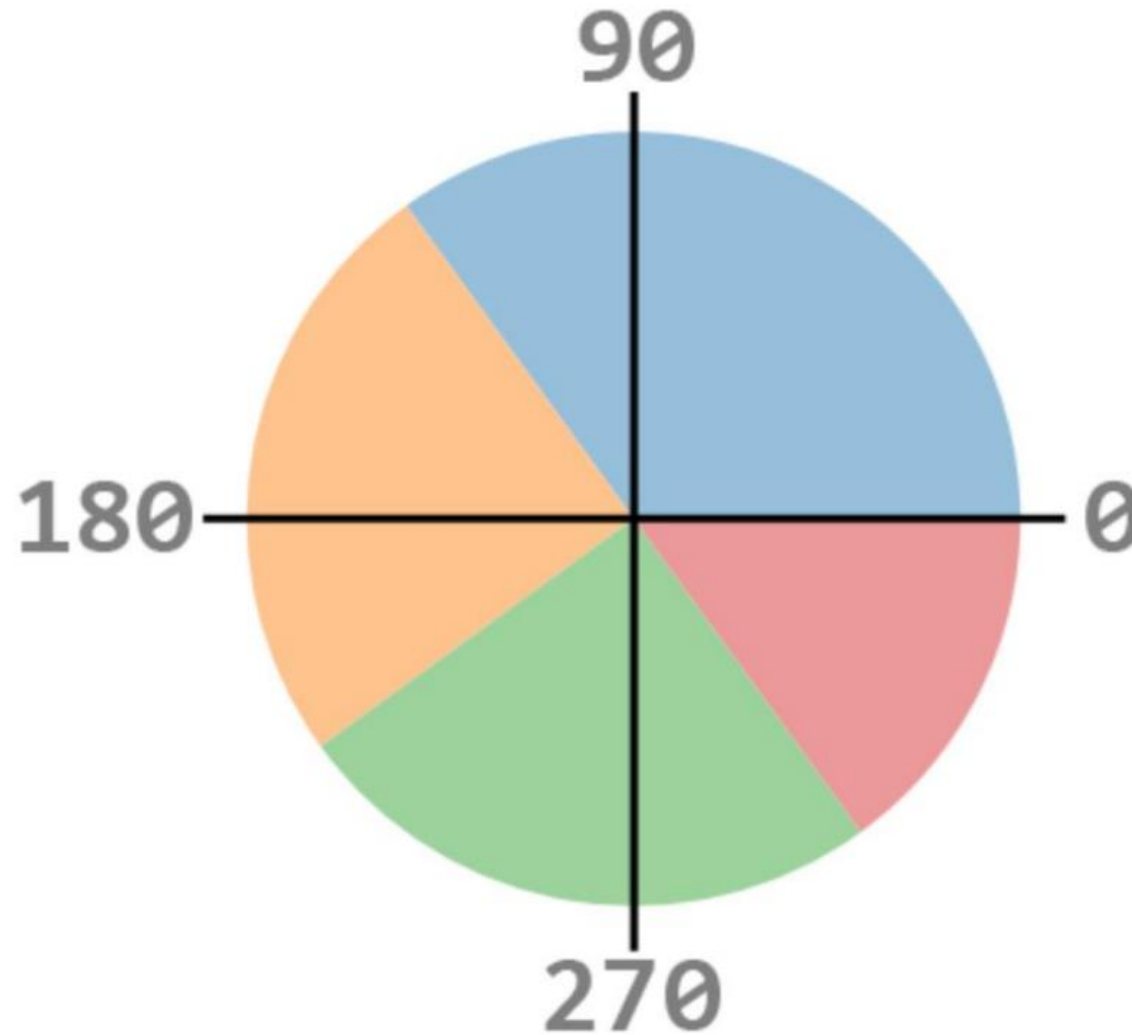
y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
plt.show()
>>
```



Start Angle

As mentioned, the default start angle is at the x-axis, but you can change the start angle by specifying a `startangle` parameter. The `startangle` parameter is defined with an angle in degrees, default angle is 0:



Example:

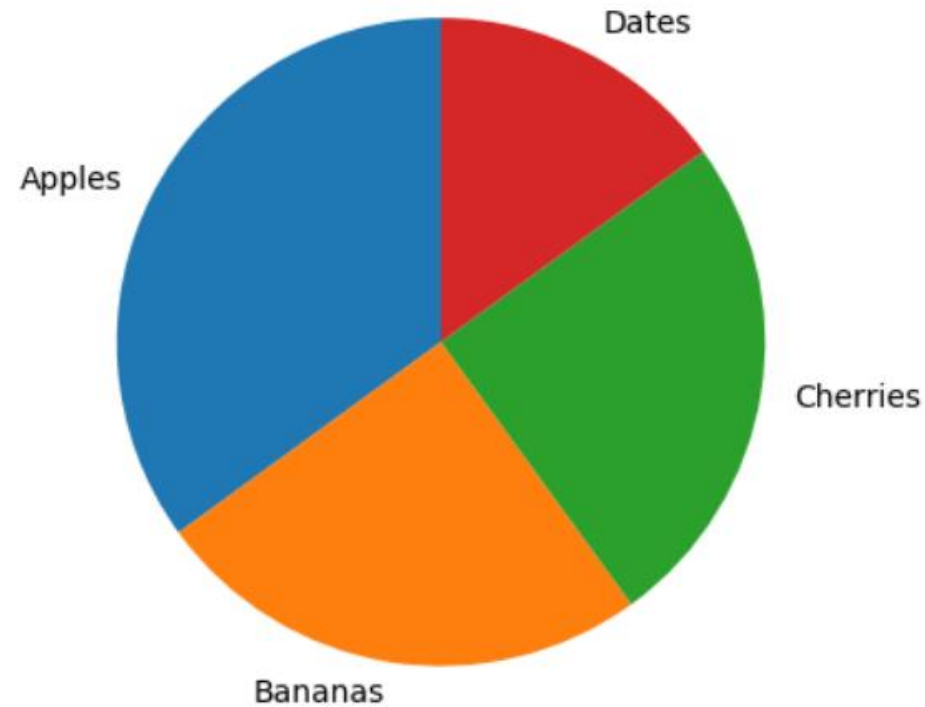
Start the first wedge at 90 degrees:

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
y = np.array([35, 25, 25, 15])  
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
```

```
plt.pie(y, labels = mylabels, startangle = 90)  
plt.show()
```

```
>>
```



Explode

If you want to highlight a specific wedge in your pie chart, you can use the explode parameter. The explode parameter is an array that must be specified with one value for each wedge. Each value represents how far from the center each wedge is displayed. By setting a value greater than 0 for a specific wedge, that wedge will be separated from the others. For example, if we want to separate the second wedge from the center, we can set the explode parameter to `[0, 0.1, 0, 0]`. This means that the second wedge will be separated by 0.1 from the center, while the other wedges will remain in the center.

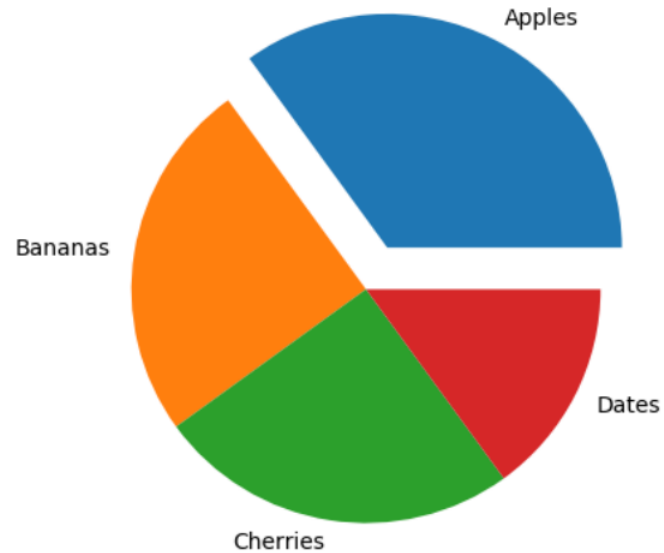
Example:

Pull the "Apples" wedge 0.2 from the center of the pie:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
myexplode = [0.2, 0, 0, 0]

plt.pie(y, labels = mylabels, explode = myexplode)
plt.show()
```



Colors

You can set the color of each wedge with the colors parameter. The colors parameter, if specified, must be an array with one value for each wedge:

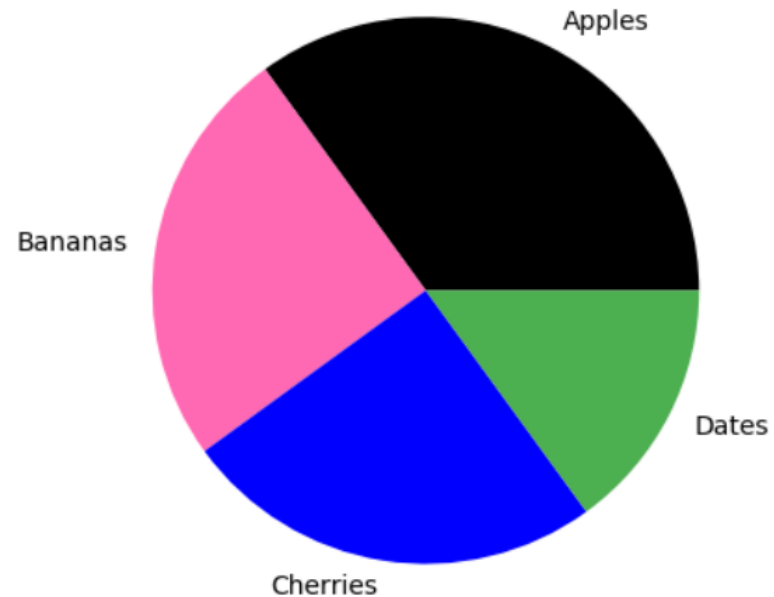
Example:

Specify a new color for each wedge:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
mycolors = ["black", "hotpink", "blue", "green"]

plt.pie(y, labels = mylabels, colors = mycolors)
plt.show()
```



Legend

To add a list of explanation for each wedge, use the `legend()` function:

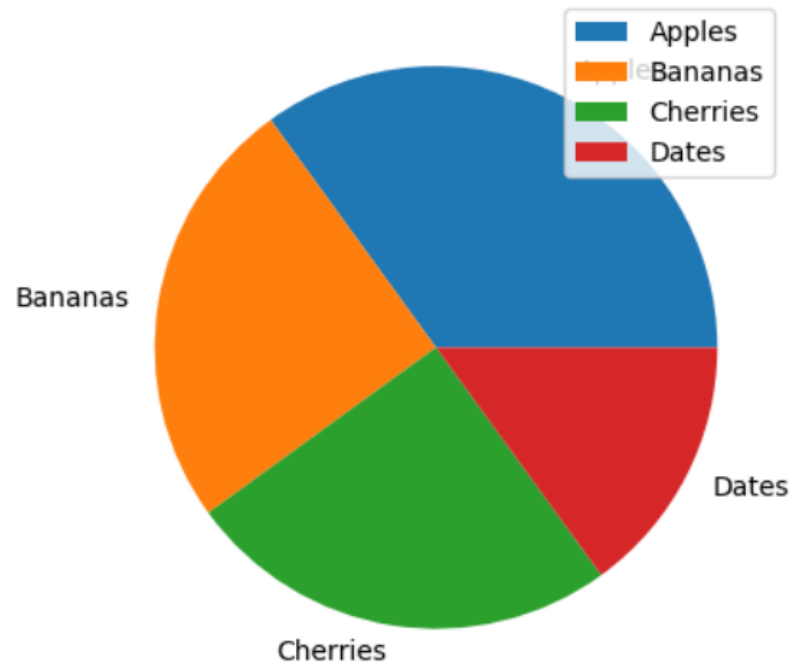
Example:

Add a legend:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
plt.legend()
plt.show()
>>
```



Thank you